

Java Application & Performance Analysis Tools

Java Backend Interview Reference

Performance Analysis Workflow

The performance analysis process: Observe symptoms -> Define metric baseline -> Profile to find bottleneck -> Fix -> Verify. Never optimize without profiling first — premature optimization wastes time.

Stage	Goal	Tools
1. Monitoring	Detect symptoms: high latency, high CPU, OOM, thread starvation	Prometheus + Grafana, Micrometer, Spring Actuator, CloudWatch
2. Profiling	Find the actual code bottleneck: which method, class, thread	JProfiler, YourKit, Async Profiler, Java Flight Recorder
3. Heap Analysis	Diagnose memory leaks, excessive allocations, GC pressure	Eclipse MAT, VisualVM, JProfiler heap snapshot
4. Thread Analysis	Find deadlocks, thread contention, blocked threads	Thread dump (jstack), JConsole, VisualVM, async-profiler
5. Load Testing	Measure throughput, latency under realistic load	JMeter, Gatling, k6, Locust
6. APM / Tracing	Distributed traces: which service/DB/call is slow	Jaeger, Zipkin, Datadog APM, New Relic, OpenTelemetry

JVM Diagnostics — Built-in Tools

jstack — Thread Dump

```
# Get thread dump (PID from jps or ps aux | grep java) jstack -l > thread_dump.txt # Look for: # BLOCKED:
thread waiting for a monitor lock (deadlock candidate) # WAITING: thread waiting indefinitely (Object.wait,
Thread.join) # TIMED_WAITING: sleeping with timeout # Multiple threads "waiting to lock <0x...>" same address
= contention # Deadlock example in dump: # "Thread-1" BLOCKED on object waiting for "Thread-2" to release it # "
Thread-2" BLOCKED on object waiting for "Thread-1" to release it
```

jmap — Heap Snapshot + Histogram

```
# Heap histogram (no dump file needed, low overhead) jmap -histo:live | head -30 # Output: num objects, total
bytes, class name # Large counts of byte[], char[], String = possible memory leak # Full heap dump (for MAT
analysis) jmap -dump:live,format=b,file=heap.hprof # NOTE: -dump causes JVM pause. Use only on non-production
or during low traffic.
```

jstat — GC Statistics

```
# Monitor GC every 1 second, 20 samples jstat -gcutil 1000 20 # Columns: S0 S1 E O M YGC YGCT FGC FGCT GCT #
S0/S1: Survivor spaces %. E: Eden %. O: Old gen %. M: Metaspace % # YGC: Young GC count. FGC: Full GC count.
GCT: Total GC time (seconds) # Warning signs: 0 approaching 100%, frequent FGC, high GCT
```

jcmm — All-in-One Diagnostic

```
jcmm help # list all commands jcmm Thread.print # thread dump jcmm GC.heap_info # heap summary jcmm GC.run
# trigger GC jcmm VM.flags # JVM flags in use jcmm JFR.start name=rec duration=60s filename=rec.jfr # start
Flight Recorder
```

Java Flight Recorder (JFR) + Mission Control (JMC)

JFR is the production-safe, low-overhead (<1% CPU) continuous profiler built into the JDK. Records method calls, GC events, thread activity, IO, lock contention, memory allocation. JMC (Java Mission Control) provides GUI to analyze JFR recordings.

```
# Enable JFR on startup (continuous) -XX:+UnlockDiagnosticVMOptions -XX:+FlightRecorder
-XX:StartFlightRecording=duration=60s,filename=app.jfr,settings=profile # Or start dynamically jcmm JFR.start
name=myRec maxsize=250m maxage=6h jcmm JFR.dump name=myRec filename=myapp.jfr jcmm JFR.stop name=myRec # Key
JFR events to analyze in JMC: # - Method profiling: which methods consume most CPU # - GC: allocation rate, GC
pause times # - Lock instances: most contended locks # - File/Network I/O: slow calls # - Thread sleep/park:
where threads are waiting
```

Async Profiler (Production Profiler)

Async Profiler is a low-overhead sampling profiler that uses OS perf_events/AsyncGetCallTrace. Unlike JFR, it captures CPU time correctly without safepoint bias. Generates flame graphs.

```
# Download: https://github.com/async-profiler/async-profiler ./profiler.sh -d 30 -f flamegraph.html # 30 sec
CPU profile -> HTML flame graph ./profiler.sh -e alloc -d 30 -f alloc.html # allocation profile ./profiler.sh
-e lock -d 30 -f lock.html # lock contention profile # Reading a flame graph: # X-axis = % of total CPU time
(wider = more time) # Y-axis = call stack (bottom = thread entry, top = leaf method = doing actual work) #
Look for: wide bars near the top = hot methods to optimize # Flat tops = methods that return quickly # Self
time = time in method itself vs calling children
```

Heap Analysis — Eclipse MAT

Eclipse Memory Analyzer Tool (MAT) analyzes heap dumps (.hprof) to find memory leaks and excessive object retention.

MAT Feature	What It Shows	How to Use
Dominator Tree	Objects that retain the most heap. Top entries = likely leak suspects.	Open Dominator Tree view; expand top entries to see retained objects
Leak Suspects	Automatic analysis pointing to likely memory leaks with explanation	Run report: Leak Suspects. MAT shows accumulation point.
Histogram	Count and size of each class. Large counts of domain objects = leak.	Sort by retained heap; filter by package name
OQL	SQL-like Object Query Language to filter and aggregate objects	<code>SELECT * FROM java.util.HashMap WHERE size > 10000</code>
Retained Heap	Total memory freed if this object is GC'd (including all it references)	High retained heap on non-framework class = leak
GC Roots	Show what holds an object from being GC'd	Right-click object -> Merge Shortest Paths to GC Roots

VisualVM

Free, bundled with JDK (jvisualvm). Good for development-time profiling. Connects to local or remote JVM via JMX.

Tab	What You Can Do
Overview	JVM args, system properties, PID, main class
Monitor	Live CPU, heap, threads, classes charts. Trigger GC. Heap dump button.
Threads	Live thread state (Running/Sleeping/Waiting/Blocked). Thread dump button.
Profiler	CPU profiling: which methods take most time. Memory profiling: allocation by class.
Sampler	Low-overhead sampling (better for production-like profiling). CPU and memory sampling.
Heap Dump	Generate and analyze heap dump inline. Basic MAT-like functionality.

APM Tools — Distributed Performance

Tool	Type	Key Features
Jaeger	Open-source distributed tracing	Trace requests across microservices. OpenTelemetry compatible. See latency per service/DB call.
Zipkin	Open-source distributed tracing	Simpler than Jaeger. Good for Spring Cloud Sleuth integration.
Datadog APM	Commercial APM	Auto-instrumentation, flame graphs, anomaly detection, log correlation. Full observability platform.
New Relic	Commercial APM	JVM metrics, transaction traces, error analysis, infrastructure monitoring.
OpenTelemetry	Vendor-neutral standard	Instrument once, export to any backend (Jaeger, Zipkin, Datadog, etc). Java agent or SDK.
Elastic APM	APM on Elasticsearch	Integrates with ELK stack. Distributed traces, metrics, logs in one platform.

Load Testing Tools

Tool	Language	Best For
Apache JMeter	Java (GUI + XML scripts)	Enterprise load testing. Complex scenarios, assertions, distributed testing. GUI makes it accessible.
Gatling	Scala DSL	Code-first, high performance, built-in HTML reports. CI/CD integration. Best for Java shops.
k6	JavaScript	Developer-friendly, CI-native. Grafana integration. Good for REST and WebSocket testing.
Locust	Python	Distributed, scriptable. Easy for complex user behavior simulation.
wrk / hey	CLI tools	Micro-benchmarking: max throughput of a single endpoint. Not for complex scenarios.

Gatling Example (Spring Boot endpoint)

```
class OrderSimulation extends Simulation { val httpConf = http.baseUrl("http://localhost:8080") val scn =
scenario("Get Orders") .exec(http("Get Orders").get("/api/orders").check(status.is(200))) setUp(scn.inject(
rampUsersPerSec(10).to(100).during(60.seconds), // ramp from 10 to 100 RPS over 1 min
constantUsersPerSec(100).during(120.seconds) // sustain 100 RPS for 2 min ).protocols(httpConf)) .
assertions(global.responseTime.p99.lt(200)) // p99 latency < 200ms }
```

Micrometer + Spring Actuator Metrics

```
# application.yml management: endpoints.web.exposure.include: "*" metrics.export.prometheus.enabled: true //
Key JVM metrics exposed at /actuator/prometheus: // jvm_memory_used_bytes{area="heap"} - heap usage //
jvm_gc_pause_seconds_sum - GC pause time // jvm_threads_live_threads - thread count //
http_server_requests_seconds - request latency per endpoint // hikaricp_connections_active - DB connection
pool usage // system_cpu_usage - CPU // Custom metrics @Autowired MeterRegistry registry; Counter orderCounter
= registry.counter("orders.created", "status", "success"); Timer dbTimer = registry.timer("db.query.duration", "
query", "findById"); Gauge.builder("queue.size", queue, Queue::size).register(registry);
```

Common Performance Problems & Diagnoses

Symptom	Likely Cause	Tool / Fix
High CPU, slow response	Hot method in tight loop, inefficient algorithm, excessive serialization	Async Profiler / JFR flame graph -> find hot method
OutOfMemoryError: Heap	Memory leak (objects retained forever), too-large allocations	jmap heap dump -> MAT Dominator Tree / Leak Suspects
OutOfMemoryError: Metaspace	Class loader leak (common in dynamic classloading, hot reload)	jstat -gcmetacapacity; check classloader count in VisualVM
Thread starvation / timeout	Thread pool too small, thread pool exhausted, blocking I/O in reactive context	jstack thread dump; look for BLOCKED threads; increase pool or add timeout
Frequent Full GC	Old gen memory leak, oversized objects promoted to old gen, short-lived large allocations	jstat -gcutil; JFR GC events; MAT heap analysis
High DB latency	Missing index, N+1 queries, connection pool exhausted, lock contention	Slow query log; EXPLAIN; hikari metrics; APM trace
High p99 latency, normal avg	GC pauses affecting tail latency, lock contention spikes	JFR lock events; async-profiler -e lock; GC log analysis
Memory grows slowly over time	Memory leak: cache without eviction, ThreadLocal not cleared, listener not removed	Take heaps 1h apart; compare dominator trees; look for growing collections

GC Tuning Essentials

```
# Common GC flags -XX:+UseG1GC # G1 (default Java 9+): good for low-pause -XX:+UseZGC # ZGC (Java 15+):
sub-millisecond pauses, large heaps -XX:+UseShenandoahGC # Shenandoah: low pause, Red Hat -Xms2g -Xmx8g #
min/max heap (set equal in prod to avoid resize pauses) -XX:MaxGCPauseMillis=200 # G1: target max GC pause
(soft goal) -XX:+PrintGCDetails # Log GC details -Xlog:gc*:file=gc.log # Java 9+ unified GC logging # GC log
```

analysis tools: # GCViewer: visualize GC log # GCEasy: online GC log analyzer (gcEasy.io) # Rule of thumb: GC time < 5% of wall-clock time is acceptable

Interview Q&A;

Q1: What is the difference between CPU profiling and sampling profiling?

CPU profiling (instrumentation) inserts probes at every method entry/exit to measure exact time. High accuracy but significant overhead (10-30%), changes behavior. Sampling profiling takes periodic stack traces (e.g., every 10ms) and counts how often each method appears. Lower overhead (~1-2%) and safe for production. Async Profiler uses sampling with OS perf_events to avoid safepoint bias (JFR/JVMTI profilers can only sample at safepoints, which skews results).

Q2: How do you find a memory leak in a Java application?

1. jstat -gcutil: check if Old Gen grows over time without GC recovery. 2. Take heap dumps 1 hour apart: jmap -dump:live,format=b,file=heap1.hprof pid. 3. Open both in Eclipse MAT. 4. Use Dominator Tree to find objects retaining most heap. 5. Use Leak Suspects report. 6. Compare histograms between the two dumps — what grew? Common causes: unbounded caches (HashMap), static references, ThreadLocal not cleared, event listeners not removed.

Q3: What is safepoint bias in profiling?

JVMTI-based profilers (VisualVM, JProfiler) can only take stack traces when the JVM is at a safepoint — a specific point in execution where the JVM can safely inspect thread state. JVM only reaches safepoints at method boundaries, loop back edges — so methods that do long work between safepoints are underrepresented. Async Profiler uses Linux perf_events (OS-level) or AsyncGetCallTrace to sample regardless of safepoints, giving more accurate results.

Q4: What does a thread dump tell you and how do you read it?

A thread dump (jstack) shows every thread's state and stack trace at that instant. Key states: RUNNABLE = executing or waiting for CPU; BLOCKED = waiting to acquire a monitor lock (potential deadlock/contention); WAITING = waiting indefinitely (Object.wait, LockSupport.park); TIMED_WAITING = waiting with timeout (Thread.sleep, wait(n)). Look for: many threads BLOCKED on same object address (contention), two threads each blocked waiting on what the other holds (deadlock — jstack auto-detects and reports this).

Q5: How would you diagnose high p99 latency with normal average latency?

High p99 with low average is a "tail latency" problem. Common causes: (1) GC pauses — check jstat FGC frequency and GCT; JFR GC pause events; (2) Lock contention spikes — async-profiler -e lock or JFR LockInstance events; (3) DB connection pool exhaustion — hikaricp_connections_pending metric; (4) Thread pool saturation — thread pool metrics; (5) OS scheduling jitter — rare but look for JVM pause not attributed to GC. Fix GC with tuning or ZGC; fix contention by reducing lock scope.

Q6: What is Java Flight Recorder and how is it different from traditional profilers?

JFR is a continuous, production-safe profiler built into the JVM with <1% overhead. It records events from the JVM, JDK, and application (GC, thread activity, IO, method profiling, exceptions). Unlike external profilers that attach on demand, JFR can run continuously in production so you always have data when an incident occurs. Use jcmd to start/stop/dump recordings. Analyze with JMC (Java Mission Control). It replaces JVM-level profiling for production diagnostics.

Q7: What Micrometer metrics do you check for a Spring Boot performance investigation?

Key metrics from /actuator/prometheus: jvm_memory_used_bytes{area="heap"} — heap trend; jvm_gc_pause_seconds_sum — GC pause accumulation; jvm_threads_live_threads — thread count trend; http_server_requests_seconds (p99, p95) — latency per endpoint; hikaricp_connections_active vs hikaricp_connections_max — pool saturation; hikaricp_connections_pending — requests waiting for connection; executor_pool_size and executor_active_threads — thread pool saturation; process_cpu_usage — CPU utilization.

Q8: How do you use Gatling for load testing a REST API?

Define a simulation class extending Simulation: configure httpConf with base URL and default headers, define a scenario with exec(http().get/post with checks), set up inject() with ramp (gradually increase RPS) and constant phases, add assertions (response time p99 < threshold, success rate > 99%). Run via mvn gatling:test. Gatling generates an HTML report with latency distribution, RPS chart, and error summary. Fail the CI build if assertions fail.